

Unit Test Report

Hospital Management System

Project Name	Hospital / Healthcare Management System
Module Tested	Backend Service Layer (healthcare-backend)
Technology Stack	Java 17 LTS, Spring Boot 3, Maven
Testing Frameworks	JUnit 5 (Jupiter), Mockito 5
Testing Type	Unit Testing (Service Layer)
Test Environment / IDE	VS Code / IntelliJ IDEA / Windows PowerShell
Report Date	08-09-2026

1. Objective

The objective of this report is to document the unit testing carried out on the service layer of the Hospital Management System backend. The tests were performed to ensure that the main business logic works correctly and independently of the database, controllers, and other external components.

The testing also helped identify errors early, verify proper exception handling, and ensure that key functions such as patient registration, doctor management, prescriptions, medical records, and appointment scheduling work as expected.

2. Scope of Testing

Testing was restricted to the Service Layer of the application, as this is where core business logic and domain validations reside. The following four service classes were covered:

Service Class	Responsibility Tested
AppointmentService	Booking, double-booking prevention, patient conflict validation, cancellation, rescheduling, and doctor clinical prescription recording.
UserService	User registration with role-based sub-entity creation (Doctor/Patient with shift & hours), login authentication, and fallback defaults.
DoctorService	Doctor record management, availability status updates, consultation fee adjustments (₹ INR), and shift/working hour updates.
PatientService	Patient record management, patient directory retrieval, and medical history updates.

3. Tools & Technologies Used

Tool / Library	Purpose
JUnit 5 (Jupiter)	Test execution framework — defines test cases, assertions, and lifecycle (@Test, @BeforeEach, @DisplayName).
Mockito 5	Mocking framework used to mock repository dependencies to isolate service logic from the database.
Maven (Surefire Plugin)	Dependency management and automated test execution build lifecycle (./mvnw test).
Java 17 LTS	Modern Long-Term Support Java runtime for backend execution and testing.

4. Testing Methodology

Each service class was tested using the Mockito mocking framework in combination with JUnit 5. Repository interfaces (AppointmentRepository, UserRepository, DoctorRepository, PatientRepository) were mocked using Mockito.mock() and injected into the service class under test. This approach ensures that only the service business logic is tested without triggering database side effects.

For every method, both the success (happy path) scenario and relevant failure/exception scenarios were verified (e.g., booking conflicts, missing records, invalid credentials, and default-value fallback logic during registration).

Every test adheres to the AAA (Arrange-Act-Assert) pattern: Arrange prepares input data and mock expectations; Act executes the target service method; and Assert validates output values and exception boundaries.

5. Test Case Details

The tables below detail all 32 unit test cases written and executed across the four service classes, grouped by service module.

5.1 AppointmentServiceTest (10 test cases)

Covers appointment booking, double-booking prevention, patient conflict validation, cancellations, rescheduling, and doctor prescriptions.

#	Test Case Name	Scenario / Purpose	Result
1	testCreateAppointment_Success	Books an appointment successfully when patient, doctor, date, and time slot are all valid and free.	PASS
2	testCreateAppointment_PatientNotFound_ThrowsException	Throws an error when booking with a non-existent patient ID.	PASS
3	testCreateAppointment_DoctorNotFound_ThrowsException	Throws an error when booking with a non-existent doctor ID.	PASS
4	testCreateAppointment_MissingDate_ThrowsException	Throws an error when appointment date is blank or null.	PASS
5	testCreateAppointment_MissingTime_ThrowsException	Throws an error when appointment time is blank or null.	PASS

6	testCreateAppointment_DoctorDoubleBooking_ThrowsException	Prevents double-booking the same doctor at the same date and time slot.	PASS
7	testCreateAppointment_PatientDoubleBooking_ThrowsException	Prevents a patient from booking two overlapping appointments at the same time.	PASS
8	testCancelAppointment_Success	Cancels an existing appointment and updates its status to CANCELLED.	PASS
9	testRescheduleAppointment_Success	Reschedules an appointment to a new date/time slot when the slot is free.	PASS
10	testUpdatePrescription_Success	Saves doctor's clinical observations and prescribed medication instructions on the appointment.	PASS

5.2 UserServiceTest (10 test cases)

Covers user registration (role-based sub-entity creation), fallback defaults, user retrieval, and login authentication.

#	Test Case Name	Scenario / Purpose	Result
1	testRegisterPatient_Success	Registers a user with role PATIENT and creates a linked Patient record.	PASS
2	testRegisterDoctor_WithShiftAndHours	Registers a user with role DOCTOR and stores shift (Day/Night) and working hours.	PASS
3	testRegisterDoctor_AppliesDefaultsWhenFieldsBlank	Applies default values (specialization, fee, availability) when doctor fields are left blank.	PASS
4	testRegisterUser_DefaultRolePatientWhenNull	Defaults role to PATIENT when no role is explicitly provided in the request.	PASS
5	testSaveUser_PatientRole_CreatesPatientIfNotExists	Auto-creates a Patient record when saving a PATIENT user who has none.	PASS
6	testSaveUser_DoctorRole_CreatesDoctorIfNotExists	Auto-creates a Doctor record when saving a DOCTOR user who has none.	PASS
7	testLogin_Success	Authenticates successfully when given matching email and password.	PASS
8	testLogin_InvalidPassword_ThrowsException	Throws an error when the password does not match.	PASS
9	testLogin_UserNotFound_ThrowsException	Throws an error when no user exists with the given email.	PASS
10	testGetAllUsers_ReturnsUserList	Retrieves the complete list of registered users for the admin portal.	PASS

5.3 DoctorServiceTest (8 test cases)

Covers doctor record management: creation, retrieval, availability, consultation fees, and shift updates.

#	Test Case Name	Scenario / Purpose	Result
1	testSaveDoctor_Success	Saves a new doctor record and returns the saved entity.	PASS
2	testGetAllDoctors_ReturnsList	Retrieves the complete list of doctors across hospital departments.	PASS
3	testUpdateAvailability_Success	Updates a doctor's availability status successfully (e.g. Available to Busy).	PASS
4	testUpdateAvailability_DoctorNotFound_ThrowsException	Throws an error when updating availability for a non-existent doctor ID.	PASS
5	testUpdateConsultationFee_Success	Updates a doctor's consultation fee successfully in Indian Rupees (₹).	PASS
6	testUpdateConsultationFee_DoctorNotFound_ThrowsException	Throws an error when updating fee for a non-existent doctor ID.	PASS
7	testUpdateShiftAndWorkingHours_Success	Updates a doctor's shift (Day/Night) and daily working hours.	PASS
8	testUpdateShift_DoctorNotFound_ThrowsException	Throws an error when updating shift for a non-existent doctor ID.	PASS

5.4 PatientServiceTest (4 test cases)

Covers patient record management: creation, retrieval, and medical history updates.

#	Test Case Name	Scenario / Purpose	Result
1	testSavePatient_Success	Saves a new patient record and returns the saved entity.	PASS
2	testGetAllPatients_ReturnsList	Retrieves the complete list of registered patients.	PASS
3	testUpdateMedicalHistory_Success	Updates a patient's medical history successfully.	PASS
4	testUpdateMedicalHistory_PatientNotFound_ThrowsException	Throws an error when updating medical history for a non-existent patient ID.	PASS

6. Test Execution Summary

Service Class	Test Cases	Passed	Failed
AppointmentServiceTest	10	10	0
UserServiceTest	10	10	0
DoctorServiceTest	8	8	0
PatientServiceTest	4	4	0
Total	32	32	0

Overall Result: 32 / 32 test cases passed (100% pass rate), with 0 failures and 0 errors, executed in approximately 21.39 seconds.

6.1 Execution Evidence (Maven Surefire Test Runner)

The log output below was captured directly from the Maven test execution panel after executing all service test classes:

```
[INFO] -----
[INFO] T E S T S
[INFO] -----
[INFO] Running com.healthcare.healthcare_backend.service.AppointmentServiceTest
[INFO] Tests run: 10, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.953 s
[INFO] Running com.healthcare.healthcare_backend.service.DoctorServiceTest
[INFO] Tests run: 8, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.112 s
[INFO] Running com.healthcare.healthcare_backend.service.PatientServiceTest
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.058 s
[INFO] Running com.healthcare.healthcare_backend.service.UserServiceTest
[INFO] Tests run: 10, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.241 s
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 32, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 21.394 s
```

7. Conclusion

- **Double-Booking & Slot Conflict Prevention:** The conflict check in AppointmentService detects occupied slots and throws AppointmentAlreadyExistsException before database persistence, preventing doctor schedule overlap.
- **Patient Concurrency Safeguard:** Patients are prevented from scheduling multiple simultaneous consultations with different doctors at the exact same time slot.
- **Role-Based Entity Creation:** UserService differentiates between PATIENT and DOCTOR registrations, attaching medical records or doctor parameters (specialization, fee in ₹, shift timing, and working hours).
- **Prescription Workflow:** The prescription update method writes doctor clinical notes and medication instructions directly onto the appointment record without disrupting scheduling states.
- **Exception Handling & Validation:** All negative test cases (wrong password, user not found, missing date, missing time, record not found) triggered appropriate domain exceptions without uncaught crashes.
- **Zero Failures:** All 32 unit tests executed cleanly with a 100% pass rate under automated Maven test execution.